Sprint 8 · Reto «Duelo de modelos»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. TurisData ya sabe que se pueden predecir las cancelaciones de reservas (lo visteis en el Sprint 7). Ahora el gerente pregunta: **¿qué modelo ponemos en producción?** Duda entre tres cosas: *«¿el más preciso? ¿el que yo pueda entender? ¿el más barato de mantener?»*. Vuestro equipo tiene que **enfrentar varios modelos** (árboles, bosques, *boosting* y SVM), medirlos todos con las mismas reglas en una tabla de resultados (el *leaderboard*) y **elegir un campeón con argumentos**. La idea clave: en la vida real **el modelo más preciso no siempre es el mejor**; uno un poco peor, pero que se entiende y se mantiene fácilmente, puede ser la mejor decisión. Por eso se decide con **tres criterios**: rendimiento, explicabilidad y coste.

Cómo se trabaja. No hace falta experiencia previa: el cuaderno `S08_06` ya trae las herramientas para medir todos los modelos igual, y vosotros rellenáis los huecos paso a paso. Hay una regla de oro: **el test se usa una sola vez, solo con el campeón**; probar varios modelos en el test y quedarse con el mejor es hacer trampa. El trabajo completo son unos **150 minutos de equipo** (cinco pasos): en clase tenéis 10 minutos de planning (día 1), **75 minutos de reto + 35 de entrega, review y retro** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (el navegador, sin instalar nada) con las librerías `scikit-learn`, `xgboost` y `lightgbm`	Ejecutar el cuaderno del reto	Ya instaladas en Colab
El cuaderno del reto `S08_06_leaderboard_plantilla`	Medir todos los modelos igual, guardar `leaderboard_S08.csv`, calcular la puntuación global y evaluar el campeón en el test	Colab. Su código explicado
Cuadernos de modelos: `S08_01` árboles, `S08_02` bosques, `S08_03` *boosting* y `S08_04` SVM	Las pistas para cada familia de modelos	Colab; su código explicado
Cuaderno de búsqueda de hiperparámetros `S08_05`	La guía para el paso 3 (`GridSearchCV` y `RandomizedSearchCV`). Es el cuaderno **del reto** que se hace **en casa la tarde del 11/11** (≈ 40 min; para el paso 3 os bastan las secciones 2 y 3): venid con él hecho al paso 3	Colab; su código explicado
Presentaciones y apuntes	Repasar cómo funciona cada modelo: árboles, ensembles, SVM y apuntes	Carpeta `1_Teoria`

Recurso	Para qué lo usaréis	Dónde está
Plantilla de la *model card* `model_card_plantilla.md`	La «ficha técnica» del campeón, con sus apartados	En GitHub: *Raw* y copiar el texto
Plantilla del *leaderboard* `leaderboard_plantilla.csv`	Ver qué columnas tiene la tabla (el cuaderno ya crea la vuestra)	Solo para consultar
Los datos `reservas_turisdata.csv`	El problema: predecir la columna `cancelada`. Son reservas **sintéticas**	El cuaderno los descarga solo si no los encuentra
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-08/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender modelos y errores, y preparar la defensa	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son cinco pasos, de **150 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. El calendario da menos tiempo de reto del que suman los pasos: el día 1 solo hay 10 minutos (el planning, donde se acuerdan los pesos; los otros 10 son para escribirlos justificados al empezar el día 2) y el día 2 hay 75 minutos de reto, más la entrega, la review y la retro. Los 65 minutos que faltan se hacen al final del día 2, en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30); la subida puede hacerse hasta esa hora. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Planning y pesos de los criterios (día 1, 20 minutos)

1. Repartir roles y tareas, y apuntarlas en el tablero.
2. Leer el reto y decidir hasta qué nivel queréis llegar (, o).
3. Acordar los **pesos** de los tres criterios con la «voz del cliente» (el docente) y justificarlos.
4. Escribir los pesos y su justificación en un sitio común: los usaréis en los pasos 4 y 5.

En sencillo. Imaginad que vuestra empresa tiene que elegir una furgoneta. Una es la más potente, otra es la que cualquiera sabe conducir y arreglar, y otra es la más barata de mantener. No hay una «mejor» a secas: depende de lo que le importe más a quien paga. En este reto hacéis lo mismo con modelos de inteligencia artificial que avisan de qué reservas se van a cancelar: los enfrentáis en una tabla de clasificación (el *leaderboard*) y elegís un campeón con **tres criterios**. En este primer paso **no se programa nada**: decidís cuánto vale cada criterio (los **pesos**) junto con el cliente, que es el docente, y repartís el trabajo. No hace falta saber qué es un árbol, un bosque o una SVM: los iréis conociendo en los pasos siguientes.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Modelo	Un programa que aprende de reservas pasadas a calcular la probabilidad de que una reserva nueva se cancele.
Leaderboard	La tabla de clasificación: una fila por modelo, con sus notas. Es un fichero, `leaderboard_S08.csv`.
Peso	Cuánto cuenta un criterio en la nota final. Los tres pesos suman 100 % (en el cuaderno, 1).
AUC	Una nota de 0,5 a 1 que dice cómo de bien separa un modelo las reservas que se cancelan de las que no. 0,5 es como tirar una moneda; 1 es perfecto.
Validación cruzada	Medir el modelo varias veces con trozos distintos de los datos de entrenamiento (aquí, 5) y quedarse con la media: la nota es más fiable que con una sola prueba.

Palabra	Qué significa, en sencillo
Explicabilidad	Poder decir **por qué** el modelo avisa de una cancelación.
Coste	Lo que cuesta tener el modelo en marcha: tiempo de entrenarlo, tiempo de predecir y espacio que ocupa.
Test	Un trozo de las reservas (1200 de 6000) que se aparta y no se toca hasta el final: el examen final. Se usa **una sola vez**.
Model card	La «ficha técnica» del modelo elegido: para qué sirve, con qué datos, cómo rinde, riesgos y límites. Es un fichero de texto, `model_card.md`.

Cómo se hace

1. **Leed el reto entre todos (5 minutos).** Abrid el reto en una página y subrayad tres cosas. Una: el cliente pregunta **qué modelo poner en producción**. Dos: se entregan **cuatro ficheros** (`leaderboard_S08.csv`, `model_card.md`, `reto_S08.ipynb` y `README.md`) más una autoevaluación por persona. Tres: la **regla de oro**, el test se usa **una sola vez y solo con el campeón**.
2. **Decidir el nivel.** El equipo elige hasta dónde llega:

Nivel	Qué hay que tener, en corto
Mínimo (todos los equipos)	Pesos justificados, leaderboard con 6 modelos, campeón elegido, test una sola vez, *model card* y README cortos.
Estándar	Lo anterior, más búsqueda de hiperparámetros en dos familias de modelos, análisis de sensibilidad de los pesos, *model card* completa y README con versiones y uso de IA.
Extra	Lo anterior, más importancia de variables, varias semillas o plan de mantenimiento.

   El mínimo ya es una entrega válida. Si dudáis, apuntad el estándar como objetivo y bajad al mínimo solo si el tiempo aprieta.
3. **Entender los tres criterios.** Son los que vais a ponderar. Fijaos en que dos de las tres notas las calcula el cuaderno solo y una la ponéis vosotros:

Criterio	Qué mide	De dónde sale su nota en el cuaderno
Rendimiento	Cuánto acierta: AUC en validación cruzada, con su desviación (media ± desviación).	El AUC, llevado a una escala de 0 a 1 (el peor modelo de la tabla, 0; el mejor, 1).
Explicabilidad	Si el gerente puede entender **por qué** avisa el modelo.	Una nota de 1 (caja negra) a 5 (se lee directamente) que **ponéis vosotros**; el cuaderno la pasa a una escala de 0 a 1.
Coste	Tiempo de entrenar, tiempo de predecir y tamaño del modelo.	Los tres juntos, llevados a una escala de 0 a 1 (1 = barato). El esfuerzo de mantenimiento no se calcula: se cuenta en la *model card*.

4. **Preguntar al cliente y decidir los pesos (10 minutos).** El docente hace de cliente: preguntadle lo que haría una consultora, por ejemplo «¿necesita el gerente explicar a un cliente por qué se le avisa?», «¿hay poco presupuesto para mantener el modelo?» o «¿qué le duele más, una cancelación que no vimos venir o una falsa alarma?». Partid de un reparto como **50 % rendimiento, 30 % explicabilidad, 20 % coste** y moved los números según lo que conteste. Tres reglas:
   - Los tres pesos suman **100 %**.
   - Cada peso lleva una **razón que viene del cliente**. Vale: «damos 30 % a la explicabilidad porque el gerente necesita entender por qué se avisa». No vale: «porque sí», «porque es el ejemplo» ni «porque nos salía bien».
   - No dejéis el rendimiento casi a cero: un modelo que no acierta no sirve, y el cuaderno no admite como campeón al modelo de referencia (el que no sabe nada).

5. **Repartir el trabajo.** Un reparto posible (con 3 personas, la persona D no existe y su tarea se reparte entre las demás). **Todas las personas revisan el trabajo de otra:**

Persona	Tarea	Paso
A	Modelos de referencia, logística, árbol y bosque en el cuaderno	2
B	*Boosting*, SVM y primera búsqueda de hiperparámetros	3
C	Segunda búsqueda de hiperparámetros y análisis de sensibilidad	3 y 4
D (si hay 4)	*Model card* y README	5

   Acordad también quién es *Scrum Master* y quién responsable técnico en este sprint (los dos roles cambian en cada sprint). Para el cuaderno usaréis **una sola copia compartida** (*Compartir* → permiso de Editor) o una pantalla compartida, y **nunca dos personas en la misma celda a la vez**.

6. **Escribirlo todo en un sitio común.** Una tarjeta del tablero (Trello o similar, columnas *Por hacer* → *En curso* → *En revisión* → *Hecho*) con este texto, con vuestros datos:

```
Nivel objetivo: estándar
Pesos (rendimiento / explicabilidad / coste): 50 % / 30 % / 20 %
Por qué: damos 30 % a la explicabilidad porque el gerente necesita entender por qué se avisa.
Reparto: A = …, B = …, C = …
```

   Y una tarjeta por tarea, cada una con su responsable. Si el planning se queda corto, la justificación se termina de escribir al empezar el día 2.

Qué deberíais ver. Una tarjeta (o un mensaje fijado) con el nivel objetivo, los tres pesos que suman 100 % y una frase de por qué para cada uno, y en el tablero una tarjeta por tarea con su responsable. Nada de código ni de resultados todavía.

Hecho cuando: los pesos suman 100 %, están justificados con una razón del cliente y cada persona sabe qué le toca.

Si algo falla

Veis…	Qué significa	Qué hacer
No os ponéis de acuerdo en los pesos	Es normal: cada persona valora cosas distintas.	Preguntad al docente (el cliente) qué le importa más. Si sigue el empate, votad y apuntad la razón.
Los pesos no suman 100 %	Alguna cifra está mal.	Sumadlos: 50 + 30 + 20 = 100. En el paso 4, el cuaderno dará `AssertionError: Los pesos deben sumar 1` si no suman.
Queréis dejar el rendimiento en 0 %	Con casi nada de peso al rendimiento, el modelo de referencia (que no sabe nada) puede acabar con la mejor puntuación, y el cuaderno no lo acepta como campeón.	Dad al rendimiento un peso claro, de 20 % o más.
Queréis poner el 100 % a un solo criterio	Es válido, pero entonces no hay compromiso que discutir: gana el mejor en ese criterio.	Justificadlo con una razón del cliente, o repartid el peso entre los tres.
No sabéis qué son el AUC, el árbol o el bosque	Es normal: todavía no los habéis usado.	Basta con la tabla de palabras. Hoy solo hay que decidir; los modelos se conocen en los pasos 2 y 3.
Sois 3 personas y la tarea D se queda sin dueño	El reparto de la tabla es para 4.	Repartid la *model card* y el README entre las otras tres, y apuntadlo en la tarjeta.

Paso 2 · Primeros modelos en el leaderboard (día 2, 30 minutos)

1. Abrir `S08_06_leaderboard_plantilla` en Colab, guardar una copia y llamarla `reto_S08.ipynb`.
2. Ejecutar las celdas de preparación y de herramientas **sin cambiarlas**.
3. Registrar el modelo de referencia (*baseline*), la regresión logística, un árbol podado y un bosque.
4. Comprobar que la tabla tiene una fila por modelo.

En sencillo. Un *leaderboard* es como la clasificación de una liga: todos juegan con las mismas reglas y los resultados se apuntan en una tabla. Aquí los jugadores son **modelos** y las reglas (los mismos datos, las mismas particiones de validación cruzada y la misma nota, el AUC) vienen ya hechas en una celda que **no se toca**. Vosotros metéis a los cuatro primeros: un modelo que no sabe nada (la referencia o *baseline*, para saber si los demás merecen la pena), una regresión logística, un árbol y un bosque. Cada uno se apunta con una sola orden, `registrar(evaluar_modelo(...))`: solo escribís cuál es, cómo se llama y qué nota de explicabilidad le dais. No hace falta saber cómo funciona cada modelo por dentro: los ajustes vienen dados.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Celda	Cada caja del cuaderno. Las de código se ejecutan con el botón ▶ (o con `Mayús + Intro`).
Función	Una receta con nombre que se usa con paréntesis, como `evaluar_modelo(...)`. Lo de dentro de los paréntesis son los «ingredientes» que le pasáis.
Modelo de referencia (*baseline*)	Un modelo tonto, `DummyClassifier`, que siempre predice lo más habitual sin mirar ninguna columna. Su AUC es 0,5: es el suelo que hay que superar.
Regresión logística	Un modelo sencillo que da un peso a cada variable y los suma para calcular la probabilidad de cancelar.
Árbol podado	Una cadena de preguntas de sí o no sobre la reserva a la que se limita el tamaño (`min_samples_leaf=80`: cada hoja final reúne al menos 80 reservas) para que no memorice.
Bosque aleatorio	Muchos árboles (aquí 200) que votan. Suele acertar más, pero cuesta más explicarlo.
Familia	El tipo de modelo: `referencia`, `lineal`, `arbol`, `ensemble_bagging`… Es una etiqueta que escribís al registrar.
Semilla (`SEMILLA`)	Un número (42) que fija el azar: así todos los equipos obtienen lo mismo.
`assert`	Una línea de comprobación del cuaderno: si algo no cuadra, se para y escribe un mensaje.

Cómo se hace

1. **Abrir vuestra copia del cuaderno.** Entrad en el cuaderno `S08_06` (se abre en Colab). Arriba, *Archivo → Guardar una copia en Drive* y después *Archivo → Cambiar nombre*: llamadla `reto_S08.ipynb`. Trabajad siempre en **vuestra copia**. Pulsad *Compartir* y añadid al equipo con permiso de **Editor**. Una persona escribe y las demás miran y opinan.
2. **Ejecutar las celdas de preparación y de herramientas, por orden y sin cambiarlas.** Son cuatro, de arriba abajo: «Preparación» (la primera), la de las importaciones (`import os`, `import pickle`…), la del apartado «1 · Datos y partición» y la del apartado «2 · Herramientas del leaderboard (no modificar)». Pulsad ▶ en cada una. Las dos últimas escriben un mensaje:

   ```
   Train: 4800 · Test (intacto hasta el final): 1200
   Herramientas listas. Leaderboard en: leaderboard_S08.csv
   ```

   Quiere decir que 4800 reservas se usan para entrenar y comparar, y 1200 quedan **apartadas** para el examen final del paso 4. Ojo: la celda de herramientas **borra `leaderboard_S08.csv` cada vez que se ejecuta** (por `REINICIAR = True`), así que la tabla empieza de cero.

3. **Ejercicio 1: el modelo de referencia y la logística.** Buscad la celda del apartado «3 · Registrar el baseline y el modelo lineal». Veréis una línea `# TODO`, tres puntos `...` y varias líneas con `assert`. **Borrad solo los tres puntos** y escribid ahí:

```
lb = registrar(evaluar_modelo("baseline", "referencia",
                              hacer_pipe(DummyClassifier(strategy="prior")), 5, "Siempre predice lo más habitual"))
lb = registrar(evaluar_modelo("logistica", "lineal",
                              hacer_pipe(LogisticRegression(max_iter=1000), escalar=True), 4, "La más sencilla de
explicar"))
```

   Qué hace cada línea:

   - `lb = registrar(evaluar_modelo(...))`: `evaluar_modelo` mide el modelo y devuelve una fila de resultados; `registrar` la apunta en `leaderboard_S08.csv` y devuelve **la tabla entera**, que se guarda en `lb`. Todas las líneas de este paso empiezan por `lb =`: así `lb` siempre tiene la tabla más reciente, y el `assert` de debajo la usa.
   - Los cinco datos que lleva `evaluar_modelo`, **en este orden**: el nombre (`"baseline"`), la familia (`"referencia"`), el modelo, la nota de explicabilidad (`5`) y unas notas en texto libre.
   - `hacer_pipe(...)`: prepara los datos (rellena huecos y convierte los textos en 0 y 1) y se los pasa al modelo. **El modelo nunca va suelto**: siempre dentro de `hacer_pipe(...)`.
   - `DummyClassifier(strategy="prior")`: el modelo que siempre predice lo más habitual.
   - `LogisticRegression(max_iter=1000)`: la logística; `max_iter=1000` le deja hasta 1000 intentos para ajustarse. `escalar=True` pone las variables en la misma escala antes de usarla: la logística y la SVM lo necesitan.
   - Los nombres entre comillas (`"baseline"`, `"logistica"`) tienen que ser **exactamente esos**: el `assert` los comprueba.

   Pulsad ▶. Si todo va bien, no aparece ningún mensaje de error y debajo sale la tabla `lb` con dos filas.

4. **Ejercicio 2, primera mitad: el árbol y el bosque.** Buscad la celda del apartado «4 · Registrar los contendientes del duelo». Borrad los `...` y escribid:

```
lb = registrar(evaluar_modelo("arbol_podado", "arbol",
                              hacer_pipe(DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA)),
                              5, "Reglas fáciles de leer"))
lb = registrar(evaluar_modelo("random_forest", "ensemble_bagging",
                              hacer_pipe(RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
                                                                random_state=SEMILLA, n_jobs=2)),
                              2, "200 árboles que votan"))
```

   Qué hace cada línea:

   - Es el mismo patrón que antes. Cambian el nombre, la familia, el modelo y la explicabilidad: **5** para el árbol (se lee como una lista de reglas) y **2** para el bosque (200 árboles: no se puede leer).
   - `DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA)`: el árbol; `min_samples_leaf=80` es la poda y `random_state=SEMILLA` fija el azar.
   - `RandomForestClassifier(n_estimators=200, …)`: el bosque; `n_estimators=200` son los 200 árboles y `n_jobs=2` usa dos procesadores a la vez.
   - Los nombres `"arbol_podado"` y `"random_forest"` van tal cual, y `SEMILLA` se escribe en mayúsculas: es el nombre de la variable que creó la celda de importaciones.

   Pulsad ▶. Esta celda **acabará con un error rojo, y es normal**: el `assert` pide 6 modelos y todavía hay 4 (faltan *boosting* y SVM, que son el paso 3). Los cuatro modelos **sí se han apuntado** igualmente. Para verlos, abrid una celda nueva (botón **+ Código**), escribid `lb` y pulsad ▶.

5. **Leer la tabla.** Cada fila es un modelo y cada columna, una nota. Que cada persona del equipo sepa explicar qué mide cada una:

Columna	Qué mide
`auc_cv_media` y `auc_cv_desv`	El AUC medio de las 5 pruebas de la validación cruzada y cuánto varía de una a otra (la «±»).
`ap_cv_media`	Otra nota de acierto, parecida al AUC. La tabla la guarda; no se usa para decidir.
`seg_ajuste`	Segundos que tarda en entrenarse, por prueba.
`ms_pred_1000`	Milisegundos que tarda en predecir 1000 reservas.
`tamano_kb`	Lo que ocupa el modelo guardado, en kilobytes.
`explicabilidad_1a5`	La nota que habéis puesto vosotros.
`auc_test`	Vacía hasta el paso 4: el test se estrena una sola vez.

   Si queréis entender mejor un modelo, el cuaderno de `S08_01` (árboles) y el de `S08_02` (bosques) lo explican, pero no hace falta para este paso.

Qué deberíais ver.

- **Tras las celdas de preparación:** `Train: 4800 · Test (intacto hasta el final): 1200` y `Herramientas listas. Leaderboard en: leaderboard_S08.csv`.
- **Tras el ejercicio 1:** una tabla de dos filas. `logistica` con AUC ≈ 0,758 (± 0,012) y `baseline` con AUC 0,5 y desviación 0.
- **Tras el ejercicio 2:** el error `AssertionError: Deben estar el baseline, la logística y al menos cuatro modelos más`, y al escribir `lb`, una tabla ordenada por `auc_cv_media` con cuatro filas: `logistica` (≈ 0,758), `random_forest` (≈ 0,741), `arbol_podado` (≈ 0,716) y `baseline` (0,500). El bosque es el que más pesa: `tamano_kb` de unos 11 800 KB, frente a menos de 11 KB en los demás.
- **En el panel de archivos de Colab** (el icono de la carpeta de la izquierda; si no aparece, pulsad el icono de refrescar), el fichero `leaderboard_S08.csv`.

Los AUC son los mismos en cada ejecución (con `SEMILLA = 42`) y los tiempos y tamaños pueden variar un poco de un ordenador a otro.

Hecho cuando: el leaderboard tiene `baseline`, `logistica`, `arbol_podado` y `random_forest`, y cada persona sabe explicar qué mide cada columna.

Si algo falla

Veis...	Qué significa	Qué hacer
`NameError: name 'dato' is not defined` (o `'pd'`, `'SEMILLA'`, `'hacer_pipe'`, `'DummyClassifier'`)	No se ha ejecutado una de las celdas de preparación, o Colab se reinició y olvidó todo.	Ejecutad por orden las cuatro celdas de preparación y herramientas y después la vuestra.
`AssertionError: Deben estar el baseline, la logística y al menos cuatro modelos más`	En este paso es normal: pide 6 modelos y aún hay 4.	Seguid al paso 3 sin tocar el `assert`. Los cuatro modelos están apuntados.
`AssertionError: El leaderboard debe tener baseline y logistica`	Habéis vuelto a ejecutar la celda del ejercicio 1 cuando la tabla ya tenía más modelos.	La tabla no se ha estropeado. No repitáis esa celda; para empezar de cero, ejecutad otra vez la celda de herramientas y después el ejercicio 1.
`AssertionError: El baseline debe dar AUC ≈ 0,5`	El modelo `baseline` no es el modelo de referencia.	Comprobad que es `DummyClassifier(strategy="prior")` y que va dentro de `hacer_pipe(...)`.

Veis…	Qué significa	Qué hacer
`ValueError: All the 5 fits failed.` y, al final del mensaje, `could not convert string to float: 'estandar'`	Habéis pasado el modelo suelto, sin `hacer_pipe(...)`: el modelo no sabe leer los textos de las columnas.	Envolved el modelo: `hacer_pipe(DecisionTreeClassifier(...))`.
`SyntaxError: '(' was never closed`	Falta algún paréntesis de cierre (o sobra uno).	Contad: cada `(` necesita su `)`. Al final de cada orden se cierran dos seguidos (`))`): el de `evaluar_modelo` y el de `registrar`.
La tabla sale vacía o con menos filas de las que esperáis	Se ejecutó otra vez la celda de herramientas, que borra el CSV y empieza de cero.	Volved a ejecutar los ejercicios 1 y 2 en orden.

Paso 3 · *Boosting*, SVM y búsqueda de hiperparámetros (día 2, 25 minutos)

1. Registrar un modelo de *boosting* y una SVM.
2. Buscar hiperparámetros en **dos familias** (rejilla o aleatoria), solo con train y validación cruzada, sin pasar de 5 minutos cada búsqueda.
3. Registrar en el leaderboard los mejores modelos de cada búsqueda.
4. Comprobar que hay entre 6 y 8 modelos, de al menos 4 familias.

En sencillo. Cada modelo tiene «botones de ajuste», como la temperatura y el tiempo de un horno. Girarlos a mano es lento y, sin querer, se hace trampa. Un **buscador** prueba combinaciones de botones y se queda con la mejor, pero **examinándose siempre con trozos del entrenamiento**: usar el test para elegir sería ver el examen antes de hacerlo. La **rejilla** prueba todas las combinaciones; la **aleatoria** prueba solo unas cuantas elegidas al azar. Antes, completáis la tabla con los dos modelos que faltan, *boosting* y SVM. No hace falta entender qué hace cada botón: los valores que se prueban vienen escritos más abajo (y explicados en el cuaderno `S08_05`, que es la guía de este paso).

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Hiperparámetro	Un botón de ajuste que fijáis **antes** de entrenar (cuántos árboles, qué profundidad…). El modelo no lo aprende solo.
Boosting	Muchos árboles pequeños puestos en fila: cada uno corrige los errores del anterior. LightGBM y XGBoost son dos versiones.
SVM	Un modelo que busca la «frontera» que mejor separa las reservas que se cancelan de las que no. Es una caja negra y la más lenta de la tabla.
Rejilla (`GridSearchCV`)	Búsqueda que prueba **todas** las combinaciones de una lista de valores.
Búsqueda aleatoria (`RandomizedSearchCV`)	Búsqueda que prueba solo `n_iter` combinaciones al azar, tomadas de rangos. Explora más valores con el mismo tiempo.
Diccionario	Una lista de parejas `nombre: valor` entre llaves, como `{"clf__max_depth": [4, 8]}`. Aquí, cada ajuste con los valores que se van a probar.
`clf__`	El prefijo que llevan los nombres de los ajustes: `hacer_pipe` llama `clf` a la parte del modelo, y se pegan con **dos** guiones bajos.
`best_params_`, **`best_score_`**, **`best_estimator_`**	Lo que guarda la búsqueda al acabar: los ajustes ganadores, su AUC y el modelo ya reentrenado con ellos.

Cómo se hace

1. **Terminar el ejercicio 2: *boosting* y SVM.** Volved a la celda del apartado «4 · Registrar los contendientes del duelo». Debajo de las dos líneas del paso 2 (**no las borréis**) añadid las dos siguientes (si queréis saber más, los cuadernos `S08_03` y `S08_04` explican el *boosting* y la SVM):

```
lb = registrar(evaluar_modelo("lightgbm", "ensemble_boosting",
                              hacer_pipe(lgb.LGBMClassifier(n_estimators=100, learning_rate=0.05, num_leaves=8,
                                                            random_state=SEMILLA, n_jobs=2, verbose=-1)),
                              2, "Árboles pequeños en cadena"))
lb = registrar(evaluar_modelo("svm_rbf", "svm",
                              hacer_pipe(SVC(kernel="rbf", C=1, probability=True, random_state=SEMILLA), escalar=True),
                              1, "Caja negra, la más lenta"))
```

   Qué hace cada línea:

   - Mismo patrón que en el paso 2. Cambian el modelo y su explicabilidad: **2** para *boosting* (muchos árboles) y **1** para la SVM (caja negra).
   - `lgb.LGBMClassifier(n_estimators=100, learning_rate=0.05, num_leaves=8, …)`: el modelo LightGBM. Los números son los ajustes de partida que da el cuaderno; `verbose=-1` le dice que no escriba mensajes.
   - `SVC(kernel="rbf", C=1, probability=True, …)`: la SVM. `probability=True` hace que pueda dar probabilidades, y por eso es más lenta. **Lleva `escalar=True`**, como la logística.
   - La familia del *boosting* tiene que ser `"ensemble_boosting"`: el `assert` busca la palabra *boosting*.

   Pulsad ▶. Ahora el `assert` pasa y debajo sale la tabla con **seis modelos**. La SVM tarda unos segundos más que los demás: es normal.

2. **Preparar las búsquedas.** Debajo de la celda del ejercicio 2 y **antes** del apartado «5», pulsad **+ Código** para crear una celda nueva por búsqueda (así podéis repartirlas). Los ejemplos de abajo buscan en el bosque (familia `ensemble_bagging`) y en LightGBM (`ensemble_boosting`), igual que en `S08_05`. Si preferís otras dos familias, el patrón es el mismo, pero los nombres de los ajustes cambian: preguntad antes al docente.

3. **Búsqueda 1: el bosque, con rejilla.** En una celda nueva:

```
from scipy.stats import randint, loguniform, uniform
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

pipe_rf = hacer_pipe(RandomForestClassifier(random_state=SEMILLA, n_jobs=2))
rejilla_rf = {"clf__n_estimators": [100, 200],
              "clf__max_depth": [4, 8],
              "clf__min_samples_leaf": [10, 40]}
busqueda_rf = GridSearchCV(pipe_rf, rejilla_rf, cv=cv, scoring="roc_auc").fit(X_train, y_train)
print("Bosque · mejor AUC:", round(busqueda_rf.best_score_, 4))
print(pd.Series(busqueda_rf.best_params_))
```

   Qué hace cada línea:

   - Las dos líneas `from … import …` traen las herramientas de búsqueda (`GridSearchCV`, `RandomizedSearchCV`) y los tres «generadores de números» que se usan en la búsqueda 2. Hay que ejecutarlas una vez; se quedan en la memoria del cuaderno.
   - `pipe_rf = hacer_pipe(RandomForestClassifier(…))`: el bosque envuelto en `hacer_pipe`, **sin** los ajustes que vamos a buscar.
   - `rejilla_rf = {…}`: lo que se prueba: 2 valores de `n_estimators` (cuántos árboles), 2 de `max_depth` (cuántas preguntas seguidas, como máximo) y 2 de `min_samples_leaf` (reservas mínimas por hoja). 2 × 2 × 2 = **8 combinaciones**. Fijaos en el prefijo `clf__` de cada nombre.
   - `GridSearchCV(pipe_rf, rejilla_rf, cv=cv, scoring="roc_auc")`: prepara la búsqueda con la **misma validación cruzada** del cuaderno (`cv=cv`) y puntuando con el AUC.
   - `.fit(X_train, y_train)`: la lanza. `X_train` e `y_train` son las 4800 reservas de entrenamiento y su columna `cancelada`. **Nunca `X_test`**: sería mirar el examen.
   - `busqueda_rf.best_score_` y `.best_params_`: el mejor AUC y los ajustes que lo consiguieron; `pd.Series(…)` los pone en una lista legible.

4. **Búsqueda 2: LightGBM, aleatoria.** En otra celda nueva:

```
pipe_lgb = hacer_pipe(lgb.LGBMClassifier(n_estimators=200, subsample_freq=1,
                                         random_state=SEMILLA, n_jobs=2, verbose=-1))
distr_lgb = {"clf__learning_rate": loguniform(0.01, 0.3),
             "clf__num_leaves": randint(3, 40),
             "clf__min_child_samples": randint(10, 200),
             "clf__subsample": uniform(0.6, 0.4)}
busqueda_lgb = RandomizedSearchCV(pipe_lgb, distr_lgb, n_iter=20, cv=cv, scoring="roc_auc",
                                  random_state=SEMILLA).fit(X_train, y_train)
print("LightGBM · mejor AUC:", round(busqueda_lgb.best_score_, 4))
print(pd.Series(busqueda_lgb.best_params_).round(3))
```

   Qué hace cada línea:

   - `distr_lgb = {…}`: aquí no hay listas sino **rangos**. `randint(3, 40)` da números enteros al azar entre 3 y 39; `loguniform(0.01, 0.3)` da decimales entre 0,01 y 0,3; `uniform(0.6, 0.4)` da decimales entre 0,6 y 1,0.
   - Los cuatro ajustes: `learning_rate` (cuánto corrige cada árbol al anterior), `num_leaves` (tamaño de cada árbol), `min_child_samples` (reservas mínimas por hoja) y `subsample` (qué fracción de reservas ve cada árbol).
   - `subsample_freq=1` en el modelo es necesario para que `subsample` tenga efecto.
   - `n_iter=20`: se prueban 20 combinaciones al azar. `random_state=SEMILLA` hace que sean **las mismas 20** en cada ejecución.
   - El resto es igual que en la búsqueda 1: `cv=cv`, `scoring="roc_auc"` y `.fit(X_train, y_train)`.

5. **Guardar los mejores modelos en el leaderboard.** En otra celda nueva:

```
lb = registrar(evaluar_modelo("random_forest_ajustado", "ensemble_bagging", busqueda_rf.best_estimator_,
                              2, "Rejilla de 8 combinaciones"))
lb = registrar(evaluar_modelo("lightgbm_ajustado", "ensemble_boosting", busqueda_lgb.best_estimator_,
                              2, "Búsqueda aleatoria de 20 combinaciones"))
print(len(lb), "modelos de", lb["familia"].nunique(), "familias")
lb
```

   - `busqueda_rf.best_estimator_` es el bosque ya preparado con los mejores ajustes: se pasa **tal cual**, sin `hacer_pipe`, porque ya lo lleva dentro.
   - Los nombres nuevos (`..._ajustado`) **añaden** filas. Si usarais un nombre que ya existe (`"random_forest"`), la fila se **sustituiría**. Con nombres nuevos tenéis 8 modelos, que es el máximo: **no añadáis `xgboost`**.
   - `len(lb)` cuenta las filas y `lb["familia"].nunique()` cuenta las familias distintas.

6. **Leer el resultado con honestidad.** Comparad cada modelo ajustado con el de partida y mirad la desviación (la «±»): una mejora de milésimas, menor que la desviación, es **ruido**, no una mejora. Si no mejora, es un resultado válido y se cuenta en la *model card*. Si varias combinaciones quedan casi empatadas, elegid la más sencilla (la regla del «error estándar» del apartado 4 de `S08_05`). En el paso 4 añadiréis estos dos modelos al `catalogo`.

Qué deberíais ver.

- **Tras terminar el ejercicio 2:** seis filas, ordenadas por AUC: `logistica` (≈ 0,758), `lightgbm` (≈ 0,750), `random_forest` (≈ 0,741), `svm_rbf` (≈ 0,740), `arbol_podado` (≈ 0,716) y `baseline` (0,500). La SVM es la que más tarda por prueba (segundos; las demás, menos de uno).
- **Búsqueda 1:** `Bosque · mejor AUC: 0.7456` y, debajo, los ajustes ganadores: `clf__max_depth` 8, `clf__min_samples_leaf` 10 y `clf__n_estimators` 200. Las 8 combinaciones quedan entre 0,744 y 0,746: casi empatadas.
- **Búsqueda 2:** `LightGBM · mejor AUC: 0.7526` y cuatro ajustes con decimales (aproximadamente `learning_rate` 0,092, `min_child_samples` 62, `num_leaves` 4 y `subsample` 0,889). Como son sorteos, pueden salir valores algo distintos si la versión de las librerías no es la misma.
- **Al registrar los dos modelos ajustados:** `8 modelos de 6 familias` y una tabla de 8 filas, con `lightgbm_ajustado` (≈ 0,753) justo por encima de `lightgbm` (≈ 0,750) y `random_forest_ajustado` (≈ 0,746) sobre `random_forest` (≈ 0,741). Mejoras de milésimas: ruido, porque la desviación es de ± 0,013. Cada búsqueda tardó entre 15 y 40 segundos en un ordenador de prueba; en Colab puede tardar más.

Hecho cuando: hay entre 6 y 8 modelos de al menos 4 familias, y la búsqueda está hecha en dos familias con validación cruzada sobre train.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'GridSearchCV' is not defined` (o `'randint'`, `'loguniform'`, `'uniform'`)	Falta ejecutar las dos líneas `from … import …` del principio de la búsqueda 1.	Ejecutad esa celda (o añadidlas al principio de la que falla) y volved a probar.
`ValueError: Invalid parameter 'max_depth' for estimator Pipeline(…)`	A un nombre de la rejilla le falta el prefijo `clf__`.	Escribidlo como `"clf__max_depth"`, con dos guiones bajos.
`ValueError: Invalid parameter 'rf' for estimator Pipeline(…)` (o `'lgb'`)	Habéis copiado el prefijo de `S08_05`: allí el paso se llama `rf` o `lgb`; aquí se llama `clf`.	Cambiad el prefijo por `clf__`.
`AssertionError: Falta un modelo de boosting`	La familia del modelo no contiene la palabra *boosting*.	Usad `"ensemble_boosting"`.

Veis…	Qué significa	Qué hacer
`AssertionError: Faltan modelos de las familias árbol, bosque o SVM`	Falta alguno de `arbol_podado`, `random_forest` o `svm_rbf`, o tienen otro nombre.	Revisad los nombres, que van tal cual, y que ejecutasteis la celda entera.
La búsqueda pasa de 5 minutos	Hay demasiadas combinaciones para el ordenador de Colab.	Parad la celda (el botón de parar, junto a la celda) y reducid: menos valores en la rejilla, o `n_iter=10` en la aleatoria.
La tabla tiene 9 filas o más	Hay más de 8 modelos (por ejemplo, `xgboost` más los dos ajustados).	Ejecutad de nuevo la celda de herramientas (empieza de cero) y registrad otra vez los modelos sin `xgboost`.
El modelo ajustado sale igual o peor que el de partida	La búsqueda solo elige lo mejor **de lo que probó**, y las diferencias son de milésimas.	Es un resultado válido. Contadlo así en la *model card*.

Paso 4 · Elegir el campeón (día 2, 45 minutos)

1. Calcular la puntuación global de cada modelo con vuestros pesos y ver quién gana.
2. Probar otros pesos (análisis de sensibilidad) y anotar si el campeón cambia.
3. Evaluar al campeón en el test, **una sola vez**, y guardar el resultado.
4. Escribir tres o cuatro líneas: por qué ganó, por qué no ganó otro y qué cambiaría la elección.

En sencillo. Es como adjudicar una beca con tres notas que no valen lo mismo. La nota final de cada modelo es la media de sus tres notas (rendimiento, explicabilidad y coste) **ponderada con vuestros pesos**; el primero de la lista es el **campeón**. Si cambiáis los pesos puede ganar otro modelo, y eso también es una respuesta para el cliente: «depende de lo que valoréis». Cuando el campeón está elegido llega el **examen final**: se entrena con todo el entrenamiento y se prueba **una sola vez** con las 1200 reservas que se apartaron. Si probarais varios modelos en ese examen y os quedarais con el mejor, os habríais visto las preguntas y la nota saldría demasiado optimista. En el cuaderno ya está casi todo hecho: vosotros escribís unas pocas líneas (si os perdéis, tenéis cada celda explicada en el código explicado del cuaderno).

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Puntuación global	La nota final de un modelo, de 0 a 1: la suma de sus tres notas multiplicadas por los pesos.
Campeón	El modelo con la puntuación global más alta con vuestros pesos.
Análisis de sensibilidad	Repetir la elección con otros pesos para ver si el campeón cambia. Cada reparto de pesos es un **escenario**.
`iloc[0]`	«La primera fila» de una tabla. Como la tabla viene ordenada, es la mejor.
`catalogo`	Un diccionario del cuaderno que sabe **reconstruir** cada modelo por su nombre, para poder entrenarlo de nuevo en el examen final.
`lambda:`	Una mini-receta sin nombre. `lambda: algo` significa «cuando me pidan, devuelvo `algo`».
`predict_proba`	Pide al modelo la probabilidad de cada clase. `[:, 1]` se queda con la de «se cancela».
Sobreajuste	Cuando un modelo se aprende de memoria los datos de entrenamiento y falla con los nuevos. Se nota si el AUC del test es mucho más bajo que el de validación.

Cómo se hace

1. **Mirar el gráfico del apartado «5 · Rendimiento frente a explicabilidad y coste».** Ya está escrito: solo pulsad ▶. Cada punto es un modelo; el tamaño del punto es lo que pesa el modelo y su posición dice cuánto tarda en entrenarse (hacia la derecha, más) y cuánto acierta (hacia arriba, más). Os ayuda a ver los compromisos.

2. **Ejercicio 3: pesos, puntuación y campeón.** En la celda del apartado «6 · Puntuación global con tus pesos», borrad los `...` y escribid (con **vuestros** pesos del paso 1):

```
pesos = {"rendimiento": 0.5, "explicabilidad": 0.3, "coste": 0.2}
lb_pond = puntuacion_global(lb, pesos)
campeon = lb_pond.iloc[0]["modelo"]
```

   Qué hace cada línea:

   - `pesos = {…}`: vuestros tres pesos **en decimales** (50 % se escribe `0.5`, con punto) y con las tres claves **exactas**: `"rendimiento"`, `"explicabilidad"` y `"coste"`. Tienen que sumar 1.
   - `lb_pond = puntuacion_global(lb, pesos)`: una función del cuaderno calcula la nota de 0 a 1 de cada modelo en cada criterio y las suma con vuestros pesos. Devuelve la tabla ordenada, del mejor al peor.
   - `campeon = lb_pond.iloc[0]["modelo"]`: coge la primera fila y de ella el nombre del modelo.

   Pulsad ▶. Sale `Campeón con estos pesos: …` y una tabla. Las columnas `punt_rendimiento`, `punt_explicabilidad` y `punt_coste` son las tres notas de 0 a 1, y `puntuacion_global` es la suma ponderada. Con los pesos 0,5 / 0,3 / 0,2, la logística sale así: 0,5 × 1,000 + 0,3 × 0,75 + 0,2 × 0,98 ≈ 0,92.

3. **Leer la tabla con cuidado.** Mirad el AUC **y su desviación**: un 0,81 no es mejor que un 0,80 si las desviaciones son parecidas, porque la diferencia puede ser ruido. Si dos modelos empatan, es un resultado válido: elegid el más simple y explicadlo.

4. **Probar otros pesos (sensibilidad).** Debajo hay una celda ya escrita con tres escenarios (`escenarios`): «solo rendimiento» (0.8 / 0.1 / 0.1), «equilibrado» (0.4 / 0.4 / 0.2) y «explicabilidad primero» (0.2 / 0.6 / 0.2). Pulsad ▶ y apuntad en una celda de texto (botón **+ Texto**) una tabla con el campeón de cada escenario, con este molde:

```
| Pesos (rendimiento / explicabilidad / coste) | Campeón |
|---|---|
| 0.5 / 0.3 / 0.2 (los nuestros) | … |
| 0.8 / 0.1 / 0.1 (solo rendimiento) | … |
| 0.4 / 0.4 / 0.2 (equilibrado) | … |
| 0.2 / 0.6 / 0.2 (explicabilidad primero) | … |
```

   Debajo, una frase: ¿cambia el campeón? Si cambia, vuestra recomendación depende de lo que valore el cliente, y así hay que contarlo.

5. **Preparar el catálogo.** Bajad a la celda del apartado «7 · El examen final». Veréis la función `actualizar_test` y el diccionario `catalogo`, que ya trae los modelos de partida. **Justo después de la llave `}` que lo cierra** y antes de la línea `# TODO`, añadid una línea por cada modelo ajustado del paso 3:

```
catalogo["random_forest_ajustado"] = lambda: busqueda_rf.best_estimator_
catalogo["lightgbm_ajustado"] = lambda: busqueda_lgb.best_estimator_
```

   - `catalogo["lightgbm_ajustado"] = …`: añade una entrada nueva. El nombre entre comillas tiene que ser **exactamente el mismo que en el leaderboard**.
   - `lambda: busqueda_lgb.best_estimator_`: la receta que devuelve el modelo ganador de la búsqueda.
   - Si vuestro campeón es uno de los de partida (`logistica`, `arbol_podado`, `random_forest`, `lightgbm`, `xgboost` o `svm_rbf`), ya está en el catálogo, pero añadid las líneas igualmente por si cambiáis de campeón.

6. **El examen final: solo el campeón, una vez. Antes de ejecutar, comprobad en equipo que `campeon` es el definitivo**: después de ver el número del test no se cambia. En la misma celda, en el hueco de los `...`, escribid:

```
final = catalogo[campeon]().fit(X_train, y_train)
auc_test_campeon = roc_auc_score(y_test, final.predict_proba(X_test)[:, 1])
lb_final = actualizar_test(campeon, auc_test_campeon)
```

   Qué hace cada línea:

   - `catalogo[campeon]()`: busca la receta del campeón por su nombre y la usa para crear el modelo. `.fit(X_train, y_train)` lo entrena con **todo** el entrenamiento.
   - `final.predict_proba(X_test)[:, 1]`: la probabilidad de cancelación que da el modelo para cada una de las 1200 reservas del test, que **nunca había visto**. `roc_auc_score(y_test, …)` la compara con lo que pasó de verdad y da el AUC.
   - `actualizar_test(campeon, auc_test_campeon)`: apunta ese número en la fila del campeón de `leaderboard_S08.csv` (columna `auc_test`) y devuelve la tabla. **No evaluéis ningún otro modelo en el test.**

   Pulsad ▶. Los `assert` de debajo comprueban que el AUC está entre 0,65 y 0,85 y que **solo una fila** tiene `auc_test`.

7. **Escribir las líneas de la decisión.** En una celda de texto, con este molde y **vuestros** números:

```
Campeón: … (AUC en validación cruzada … ± …; en test …).
No ganó … porque ….
Si el cliente valorara solo …, elegiríamos ….
```

   Por ejemplo, con los números de esta ejecución (los vuestros serán distintos): *«Campeón: logistica (AUC en validación cruzada 0,758 ± 0,012; en test 0,746). No ganó lightgbm_ajustado porque su AUC (0,753) es prácticamente el mismo, pero se explica peor (2 frente a 4) y el modelo pesa unas 30 veces más (124 KB frente a 4 KB). Si el cliente valorara la explicabilidad tanto como el rendimiento (pesos 0,4 / 0,4 / 0,2), elegiríamos el árbol podado.»* Esas líneas van a la *model card* (apartado 3) en el paso 5.

Qué deberíais ver.

- **Ejercicio 3** (pesos 0,5 / 0,3 / 0,2): `Campeón con estos pesos: logistica`, con `logistica` en cabeza (≈ 0,92) y `arbol_podado` muy cerca (≈ 0,90). Detrás, a mucha distancia (≈ 0,69 y menos), los demás; la SVM, con la puntuación más baja por ser caja negra, lenta y pesada.
- **Sensibilidad:** una tabla con `logistica` para «solo rendimiento» y `arbol_podado` para «equilibrado» y «explicabilidad primero»: **el campeón cambia con los pesos**.
- **Examen final:** `Campeón: logistica · AUC en validación cruzada 0.758 · AUC en test 0.746` y una tabla `lb_final` donde `auc_test` solo tiene valor en la fila del campeón (≈ 0,7458); en el resto, `NaN`.
- El AUC de test sale algo más bajo que el de validación (aquí, unas 0,01) y entre 0,65 y 0,85. Si fuera **mucho** más bajo, sería sobreajuste. En `leaderboard_S08.csv`, `auc_test` tiene valor en **una sola fila**.
- Con otros pesos o con otros modelos ajustados, vuestro campeón puede ser otro. Los tiempos cambian de un ordenador a otro y de una ejecución a otra, así que la nota de coste y la puntuación global pueden variar unas centésimas.

Hecho cuando: hay un campeón elegido con vuestros pesos, la sensibilidad está probada y apuntada, y `auc_test` aparece en una única fila.

Si algo falla

Veis…	Qué significa	Qué hacer
`AssertionError: Los pesos deben sumar 1`	Los pesos no suman 1 (por ejemplo, habéis escrito `50` en vez de `0.5`).	Escribid decimales con punto y comprobad: `0.5 + 0.3 + 0.2 = 1`.

Veis…	Qué significa	Qué hacer
`KeyError: 'coste'` (o `'rendimiento'`, `'explicabilidad'`), o `AssertionError: Claves: rendimiento, explicabilidad y coste`	Las claves de `pesos` no son exactamente esas tres.	Copiadlas tal cual, en minúsculas y sin tildes en `coste` ni `rendimiento`.
`AssertionError: El campeón debe ser uno de los modelos y no el baseline`	Con vuestros pesos gana el modelo de referencia: casi no pesa el rendimiento.	Subid el peso del rendimiento (con 0,2 o más ya no pasa) y volved a justificarlo.
`NameError: name 'lb' is not defined` (o `'busqueda_rf'`, `'campeon'`)	No se ejecutó una celda anterior, o Colab se reinició.	Ejecutad desde arriba, por orden, y no os saltéis las celdas de las búsquedas.
`KeyError: 'lightgbm_ajustado'` (u otro nombre)	El campeón no está en `catalogo`: falta la línea del paso 5 o el nombre no coincide con el del leaderboard.	Añadid la línea con **el mismo nombre** y volved a ejecutar la celda.
`ValueError: y should be a 1d array, got an array of shape (1200, 2) instead.`	Falta `[:, 1]` detrás de `predict_proba(X_test)`.	Escribid `final.predict_proba(X_test)[:, 1]`.
`AssertionError: El AUC de test del campeón debería estar entre 0,65 y 0,85`	Un AUC cercano a 1 suele ser que habéis medido con `X_train`, que el modelo ya vio.	Usad `X_test` y `y_test` en `roc_auc_score` y entrenad solo con `X_train`, `y_train`.
`AssertionError: Solo el campeón debe tener AUC de test`	Se calculó el test para más de un modelo (por ejemplo, tras cambiar de campeón).	Ejecutad de nuevo desde la celda de herramientas, que borra el CSV, y repetid el paso. No cambiéis de campeón por haber visto ese número: es la regla de oro.

Paso 5 · *Model card*, README y entrega (día 2, 30 minutos)

1. Copiar `model_card_plantilla.md` como `model_card.md` y rellenarla con vuestros datos.
2. Escribir el `README.md`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado en el README.
4. Subir los cuatro ficheros a `sprint-08/`.

En sencillo. El trabajo técnico ya está hecho; ahora hay que **dejarlo listo para que lo use otra persona**. Pensad en un electrodoméstico: dentro de la caja va la **ficha técnica** (la *model card*: qué hace, cómo rinde, para qué no sirve) y el **manual de instrucciones** (el README: qué hay en la caja, quién lo montó y cómo ponerlo en marcha). Después, alguien del equipo que no lo escribió lo revisa con una lista de comprobación (la rúbrica), con ojos frescos. Casi no hay código: se escribe **texto** con **vuestros números**, copiados del cuaderno. No hace falta saber Markdown: la plantilla ya trae los títulos (`#`) y las tablas (`|`); solo hay que rellenar lo que falta sin borrar esas marcas. Lo único nuevo en el cuaderno son tres líneas para apuntar las versiones de las librerías.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Model card	La ficha del modelo campeón, de 1–2 páginas, para el cliente y para quien lo mantenga.
README	El «léeme» de la carpeta: qué se entrega, quién hizo qué y cómo repetirlo. Es lo primero que lee el docente.
Markdown (`.md`)	Texto normal con marcas sencillas: `#` para un título, `-` para una lista y barras verticales para separar las columnas de una tabla.

Palabra	Qué significa, en sencillo
Versión	El número de edición de una librería (por ejemplo, `1.9.1`). Con otra versión, un resultado puede cambiar un poco.
Reproducir	Que otra persona repita el trabajo con el README y obtenga lo mismo: el mismo leaderboard y el mismo campeón.
Deriva	Cuando, con el tiempo, las reservas nuevas dejan de parecerse a las de entrenamiento y el modelo acierta menos.
Falsa alarma	Avisar de que una reserva se cancelará cuando no iba a hacerlo.
Rúbrica	La tabla con que se evalúa el reto: cuatro criterios, cada uno de 1 a 4.
Commit	Guardar en GitHub un cambio con un mensaje corto y vuestro nombre. Sin pulsar **Commit changes** no se guarda nada.

Cómo se hace

1. **Dejar el cuaderno definitivo y descargar dos ficheros.** Hacedlo primero: así los números que copiéis a la *model card* son los finales. Debajo de la última celda pulsad **+ Código** y escribid:

   ```
   import sklearn, xgboost, lightgbm
   print("scikit-learn", sklearn.__version__)
   print("xgboost", xgboost.__version__)
   print("lightgbm", lightgbm.__version__)
   ```

   Qué hace cada línea:

   - `import sklearn, xgboost, lightgbm`: carga las tres librerías (el cuaderno ya las usaba; volver a cargarlas no hace daño).
   - `print("scikit-learn", sklearn.__version__)`: escribe el nombre y su versión. `__version__` lleva **dos** guiones bajos a cada lado. Las otras dos líneas hacen lo mismo con `xgboost` y `lightgbm`.

   Después, *Entorno de ejecución → Reiniciar y ejecutar todo* (y confirmad). Mirad hasta el final: ninguna celda debe dar error en rojo y la última debe escribir tres líneas del estilo `scikit-learn 1.9.1` (vuestros números dependen de Colab: apuntadlos). Por último, *Archivo → Descargar → Descargar .ipynb* y, en el panel de archivos de la izquierda (icono de carpeta), clic derecho sobre `leaderboard_S08.csv` → *Descargar*. Hacedlo **antes de cerrar Colab**: al cerrar, el CSV se pierde.

2. **Rellenar la *model card*.** Abrid la plantilla y cread `model_card.md` con vuestros datos: o bien en GitHub (en la plantilla, **Raw**, `Ctrl + A`, `Ctrl + C`; después, dentro de `sprint-08`, **Add file → Create new file**, nombre `model_card.md`, pegad y completad), o bien en un editor de texto. Cada apartado lleva **vuestros** datos: no dejéis los ejemplos ni los `…`. Para el **nivel mínimo** bastan estos apartados, en pocas líneas (la última columna es un ejemplo con los números de una ejecución real; los vuestros serán otros):

Apartado	Qué escribís	Ejemplo
1. Resumen	Qué predice el modelo, para qué decisión y quién lo usa	«Calcula la probabilidad de que una reserva se cancele, para avisar al gerente de TurisData.»
2. Modelo campeón	Familia, hiperparámetros finales y versiones	«Regresión logística (familia lineal), `max_iter=1000`, variables escaladas. scikit-learn 1.9.1.»
3. Por qué este campeón	Vuestros pesos, el resultado del campeón, el mejor rival y las líneas del paso 4	«Pesos 50 / 30 / 20. `logistica`: AUC 0,758 ± 0,012, explicabilidad 4. Mejor rival, `arbol_podado`: pierde 0,042 de AUC, pero se explica mejor (5).»
5. Rendimiento	AUC en validación cruzada (± desviación) y **AUC de test (una sola evaluación)**	«Validación cruzada 0,758 ± 0,012; test 0,746.»
6. Uso previsto y NO previsto	Para qué sirve y para qué **no** debe usarse	«No usar para denegar reservas ni para decidir sin revisión humana.»

Para el **nivel estándar**, completad también estos cinco (el apartado 3 lleva además vuestra tabla de sensibilidad del paso 4):

Apartado	Qué escribís	Ejemplo
4. Datos	Fuente, número de reservas, tasa de cancelación, variables usadas y excluidas, límites	«6000 reservas sintéticas hechas en 2024–2025, 45,9 % canceladas; 15 variables; `id_reserva` y las fechas no se usan (solo la temporada de llegada). Son datos sintéticos.»
7. Explicabilidad	Cómo se explica una predicción a un gerente	«Pesan sobre todo el depósito, la antelación, el canal y la tarifa.» (Con la logística, mirad sus coeficientes como en `S07_03`; con un árbol, sus reglas; con *boosting* o SVM, decid que es caja negra.)
8. Riesgos	El **país de origen** como variable sensible y qué pasa con una falsa alarma	«La tasa de cancelación por país va del 44 % al 47,5 %: diferencia pequeña, pero el país no debe usarse para tratar distinto a nadie.»
9. Mantenimiento	Qué mediríais en producción para ver la **deriva**, cada cuánto se reentrena y quién se encarga	«Comparar cada mes la tasa de cancelación real con la prevista; reentrenar si…; responsable: …»
10. Cómo reproducirlo	Semilla, versiones, orden de ejecución y ficheros	«`SEMILLA = 42`; versiones del paso 1; orden del cuaderno de arriba abajo.»

3. **Escribir el README.** Creadlo con **Add file → Create new file** (nombre `README.md`) dentro de `sprint-08`, o editad el que ya haya en la carpeta (icono del lápiz). No hay plantilla: usad este guion, sustituyendo los `…`. Tres cosas no pueden faltar: **qué se entrega y quién hizo qué**, **cómo se reproduce** (con versiones, `SEMILLA = 42` y orden de ejecución) y **uso de IA** (qué pedisteis y qué revisasteis):

```
# Sprint 8 · Duelo de modelos · Equipo …

## Qué se entrega
- `reto_S08.ipynb`: el cuaderno, ejecutado de arriba abajo.
- `leaderboard_S08.csv`: la tabla de modelos.
- `model_card.md`: la ficha del modelo campeón.
- `README.md`: este fichero.

## Quién hizo qué
- … (nombre): …

## Cómo se reproduce
1. Abrir `reto_S08.ipynb` en Google Colab.
2. Entorno de ejecución → Reiniciar y ejecutar todo (tarda … minutos).
3. Debe salir el mismo leaderboard y el mismo campeón: ….
Versiones: scikit-learn …, xgboost …, lightgbm …. Semilla: SEMILLA = 42.

## Uso de IA
Le pedimos …; lo revisamos …; cambiamos …. Los datos son sintéticos: no hay datos personales.
```

4. **La revisión con la rúbrica (5–10 minutos).** La hace **una persona que no escribió esa parte**. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: ejecutar el cuaderno solo con el README, contar modelos, familias y filas con `auc_test` en el CSV, leer la *model card* como si fuera el gerente, poner un nivel de 1 a 4 en cada criterio **con una frase de prueba** (por ejemplo, «3 · 7 modelos, 5 familias y `auc_test` en una sola fila») y apuntar el resultado. Sin frase de prueba, el nivel no vale. Pegad esto al final del README y rellenadlo:

```
## Revisión con la rúbrica
**Revisado por:** (nombre) · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** (una cosa concreta)
```

   Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

5. **Subir los cuatro ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-08`**; (2) *Add file → Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto («Subo la model card del duelo de modelos»; no valen «cambios» ni «asdf») y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace: un reparto posible es que A suba `reto_S08.ipynb` y `leaderboard_S08.csv`, B suba `model_card.md` y C suba `README.md`. Respetad los nombres tal cual, con la S mayúscula y sin espacios. Nada de claves ni datos personales.

Qué deberíais ver. En la carpeta `sprint-08` del repositorio, los cuatro ficheros del equipo y una hoja de autoevaluación por persona (**4 + N**); y en el historial (el icono del reloj), *commits* con nombres distintos.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README y obtendría el mismo campeón, la sección «Revisión con la rúbrica» está rellena y los cuatro ficheros están en `sprint-08/`.

Después, 35 minutos de entrega, review y retro:

- **La autoevaluación.** Cada persona rellena la suya (autoevaluación y coevaluación) y la sube a `sprint-08/`; la coevaluación va dentro del mismo fichero.

- **La review (5 minutos por equipo).** Enseñad el leaderboard (la tabla o un gráfico), el campeón y **por qué no ganó otro**, y una frase sobre qué cambiaría la elección si el cliente valorara otra cosa.
- **La retrospectiva.** ¿Nos sorprendió que un modelo sencillo compitiera con uno complejo? ¿Qué criterio pesó más en nuestra decisión? ¿Qué mediríamos en producción para saber que el modelo se degrada?

Si algo falla

Veis…	Qué significa	Qué hacer
Tras *Reiniciar y ejecutar todo*, `campeon` sale distinto o las puntuaciones cambian unas centésimas	Los tiempos (ajuste y predicción) varían de una ejecución a otra y entran en la nota de coste. Si los dos primeros están a menos de 0,03, es un empate técnico.	No cambiéis de campeón tras ver el test: escribid `campeon = "…"` (vuestro modelo) en la celda del examen final, antes de `final = …`, y contad el empate en la *model card*.
`AssertionError: Solo el campeón debe tener AUC de test`	`auc_test` tiene valor en más de una fila.	Ejecutad desde la celda de herramientas, que borra el CSV, y repetid el examen final con un solo campeón.
`FileNotFoundError: [Errno 2] No such file or directory: 'leaderboard_S08.csv'`, o el CSV no sale en el panel de archivos	La sesión de Colab se reinició o se cerró y el CSV se perdió.	Ejecutad el cuaderno entero otra vez (*Reiniciar y ejecutar todo*) y descargadlo antes de cerrar Colab.
`AttributeError: module 'lightgbm' has no attribute '_version__'` (o `'version'`)	`__version__` lleva dos guiones bajos a cada lado y habéis escrito menos.	Escribid `lightgbm.__version__`, igual con las otras dos.
Subisteis un fichero a la raíz del repositorio y no a `sprint-08`	No entrasteis primero en la carpeta.	Abrid el fichero → icono del lápiz → delante del nombre escribid `sprint-08/` → **Commit changes**.
Os falta algún fichero o el nombre es otro	La rúbrica busca `leaderboard_S08.csv` y `reto_S08.ipynb` con ese nombre exacto.	Renombrad en vuestro ordenador y subidlos de nuevo con el mismo nombre.
No sabéis qué nivel poner en un criterio	Los niveles están descritos en entrega y rúbrica.	Leed la fila del criterio y poned el nivel cuyo texto **se cumple entero**; si algo se cumple a medias, es el nivel de abajo. Escribid la prueba.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da respuestas que parecen correctas y no lo son**. Reglas:

- **La IA no decide el campeón:** lo decide el cálculo con vuestros pesos.
- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea o por un número de la *model card*, tenéis que saber explicarlo.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un modelo	«Explícame cómo funciona un bosque aleatorio con una analogía de votación de expertos, sin fórmulas»	Que podéis explicárselo al gerente
Entender un hiperparámetro	«¿Qué hace `max_depth` en un árbol? ¿Qué pasa si es muy grande?»	Que probáis valores y veis lo que ocurre (sobreajuste)

Para qué	Ejemplo de petición	Qué comprobáis
Redactar la *model card*	«Ayúdame a redactar la sección de limitaciones. Estos son los datos: *[resumen]*»	Que **todo** lo que escriba coincide con vuestros resultados
Preparar la defensa	«Hazme 5 preguntas difíciles que un gerente podría hacer sobre mi elección de campeón»	Que preparáis la respuesta con **vuestros** datos

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Comparar modelos con particiones distintas	Cada persona prueba «por su lado»	Usad el cuaderno plantilla y no cambiéis la celda de herramientas: mismos folds para todos
Ajustar hiperparámetros mirando el test	«Para ver si mejora»	Buscad solo con `X_train` y validación cruzada; `X_test` solo aparece en el paso 4
Evaluar varios modelos en el test y quedarse con el mejor	Curiosidad o prisa	El test se usa **una vez**, con el campeón ya elegido; `auc_test` en una sola fila
Elegir por el AUC más alto sin mirar la desviación	«0,81 es más que 0,80»	Si las desviaciones se solapan, la diferencia puede ser ruido: elegid el más simple
`model_card` genérica	Se copian los ejemplos de la plantilla	Cada apartado con **vuestros** datos y riesgos
Olvidar la explicabilidad y el coste	Solo se mira la precisión	Puntuadlos y ponderadlos con `puntuacion_global`
Búsquedas que duran horas	«Más combinaciones»	Máximo 5 minutos por búsqueda: menos combinaciones o `RandomizedSearchCV` con pocas iteraciones
Se pierde el leaderboard	Colab borra los ficheros al cerrar y `REINICIAR = True` vuelve a empezar de cero	Descargad `leaderboard_S08.csv` antes de cerrar y no editéis el CSV a mano
El campeón no está en `catalogo`	Se registró un modelo ajustado y no se añadió al diccionario	Añadid cada modelo ajustado a `catalogo` con el mismo nombre del leaderboard

Ayuda 3 · Si os atascáis

1. **No sabéis qué peso poner:** pensad en lo que diría el gerente. Si prioriza entender, sube la explicabilidad.
2. **La búsqueda tarda demasiado:** reducid el número de combinaciones o usad búsqueda aleatoria con pocas iteraciones.
3. **Dos modelos empatan:** es un resultado válido. Elegid el más simple y explicadlo.
4. **Falla una comprobación (`assert`) del cuaderno:** leed el mensaje, que dice qué esperaba, y repasad el ejercicio de esa celda y su código explicado. Preguntad al equipo y, después, al docente.